FIM Enterprise - Complete Deployment Guide

Overview

FIM (File Integrity Monitoring) Enterprise is a scalable, distributed file integrity monitoring system with real-time alerts, baseline management, and compliance reporting.

Current Status: Production-Ready (Python 3.8+, PostgreSQL 15+)

Prerequisites

System Requirements

- **OS:** RHEL 8.x / CentOS 8.x / Ubuntu 20.04+ (tested on RHEL 8)
- **Python:** 3.8 minimum (3.11+ recommended)
- **PostgreSQL:** 15+ (with async support)
- **Node.js:** 18+ (for frontend build only)
- **RAM:** 4GB minimum (8GB for production)
- **Disk:** 50GB minimum (consider retention policies)

Required System Packages

```
# RHEL/CentOS
sudo yum install -y python3.11 python3.11-devel postgresql15-server postgresql15-devel \
  nodejs npm gcc git nginx openssl

# Ubuntu
sudo apt-get install -y python3.11 python3.11-venv postgresql postgresql-contrib \
  nodejs npm build-essential git nginx openssl
```

Installation & Setup

1. Clone and Prepare the Repository

```
cd /usr/local/opt
git clone <FIM_REPO_URL> fim
cd fim

# Create Python virtual environment
python3.11 -m venv venv
source venv/bin/activate
pip install --upgrade pip
```

2. Install Python Dependencies

```
# Install requirements
pip install -r requirements.txt --break-system-packages

# Verify critical packages (Python 3.8 compatible)
pip list | grep -E "fastapi|sqlalchemy|pydantic|cryptography"
```

Important: Avoid packages with `list[str]`, `dict[str, Any]` type hints if Python < 3.9

3. Configure Environment

Copy and customize the environment file:

```
cp .env.example .env
nano .env
```

Required variables in `.env`:

```
env
# Database
DATABASE_URL=postgresql+asyncpg://fim_user:password@localhost/fim_db
DB_POOL_SIZE=20
DB_MAX_OVERFLOW=10

# Security
SECRET_KEY=<generate with: python -c "import secrets; print(secrets.token_urlsafe(32))">
JWT_EXPIRATION_HOURS=8
CORS_ALLOWED_ORIGINS=https://fim.example.com

# SMTP (for reports)
SMTP_SERVER=mail.example.com
SMTP_PORT=587
SMTP_USERNAME=fim@example.com
SMTP_PASSWORD=<password>
SMTP_FROM_EMAIL=fim@example.com

# Request Tracker Integration (optional)
RT_URL=https://rt.example.com
RT_USERNAME=fim_user
RT_PASSWORD=<password>
RT_API_ENDPOINT=/REST/2.0

# Logging
LOG_LEVEL=INFO
LOG_FILE=/var/log/fim-backend.log
SECURITY_LOG_FILE=/var/log/fim-security.log

# Scan Configuration
AGENT_FILE_LIMIT=200000
SCAN_TIMEOUT_SECONDS=3600
BASELINE_RETENTION_MONTHS=18

# Feature Flags
ENABLE_MFA=true
ENABLE_SSO=false
```

```
ENABLE_MTLS=false
```

Generate a strong secret key:

```
python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

4. Database Setup

Create Database & User

```
sudo -u postgres psql << 'SQL'
CREATE DATABASE fim_db ENCODING 'UTF8' LC_COLLATE 'C' LC_CTYPE 'C';
CREATE USER fim_user WITH PASSWORD 'secure_password_here';
ALTER ROLE fim_user SET search_path TO fim;
GRANT CREATE ON DATABASE fim_db TO fim_user;
\c fim_db
CREATE SCHEMA fim AUTHORIZATION fim_user;
SQL
```

Run Database Migrations

```
# Activate venv
source venv/bin/activate

# Create tables manually (alembic not yet implemented)
python3 << 'PYEOF'
import asyncio
from app.core.database import db_manager
from app.models.models import Base

async def init_db():
    async with db_manager.engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    print("☑ Database tables created")

asyncio.run(init_db())
PYEOF
```

Verify tables created:

```
sudo -u postgres psql -d fim_db -c "\dt fim.*"
```

5. Create First Admin User

```
python3 << 'PYEOF'
import asyncio
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker
from app.models.models import User
from app.core.security import hash_password
import os

DATABASE_URL = os.getenv("DATABASE_URL")
engine = create_async_engine(DATABASE_URL, echo=False)
async_session = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def create_admin():
    async with async_session() as session:
        admin = User(
            username="admin",
            email="admin@fim.local",
            password_hash=hash_password("FIMAdmin@2024!"),
            full_name="System Administrator",
            role="admin",
            is_active=True
        )
        session.add(admin)
        await session.commit()
        print("☑ Admin user created: admin / FIMAdmin@2024!")
        print("⚠  CHANGE THIS PASSWORD IMMEDIATELY IN PRODUCTION")

asyncio.run(create_admin())
PYEOF
```

6. Create Necessary Directories

```
# Logs directory
sudo mkdir -p /opt/fim/logs /var/log
sudo chown -R fim:fim /opt/fim/logs
sudo touch /var/log/fim-security.log /var/log/fim-backend.log
sudo chown fim:fim /var/log/fim-*.log
sudo chmod 0640 /var/log/fim-*.log

# Backup directory
sudo mkdir -p /opt/fim/backups /mnt/fim-backup-archive
sudo chown -R fim:fim /opt/fim/backups /mnt/fim-backup-archive

# Baseline git repo
mkdir -p /usr/local/opt/fim/baselines-git
cd /usr/local/opt/fim/baselines-git
git init --bare
chmod 755 -R .
```

7. Build Frontend

```
cd /usr/local/opt/fim/frontend
npm install --legacy-peer-deps
npm run build

# Deploy to web root
sudo mkdir -p /opt/fim/web
sudo cp -r dist/* /opt/fim/web/
sudo chown -R nginx:nginx /opt/fim/web
```

Verify:

```
ls -la /opt/fim/web/
# Should show: index.html, assets/, logo.png
```

8. Systemd Service Setup

Create `/etc/systemd/system/fim-backend.service`:

ini

```
[Unit]
Description=FIM Enterprise Backend
After=network.target postgresql.service
Wants=postgresql.service

[Service]
Type=notify
User=fim
Group=fim
WorkingDirectory=/usr/local/opt/fim

# Load environment variables from .env
EnvironmentFile=/usr/local/opt/fim/.env

# Python virtual environment
Environment="PATH=/usr/local/opt/fim/venv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin"

# Start command
ExecStart=/usr/local/opt/fim/venv/bin/uvicorn \
  app.main:app \
  --host 0.0.0.0 \
  --port 8000 \
  --workers 4 \
  --access-log \
  --log-config /usr/local/opt/fim/logging.yaml

Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable fim-backend
sudo systemctl start fim-backend
sudo systemctl status fim-backend
```

9. Nginx Configuration

Create `/etc/nginx/conf.d/fim.conf`:

```nginx
upstream fim_backend {
    server 127.0.0.1:8000;
}

server {
    listen 80;
    server_name fim.example.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name fim.example.com;

    # SSL certificates (use Let's Encrypt or your CA)
    ssl_certificate /etc/ssl/certs/fim.crt;
    ssl_certificate_key /etc/ssl/private/fim.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    # Security headers
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'" always;

    # Root for static files
    root /opt/fim/web;

    # API proxy
    location /api/ {
        proxy_pass http://fim_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_redirect off;
    }

    # SPA fallback
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Enable and test:

```
sudo nginx -t
sudo systemctl enable nginx
sudo systemctl restart nginx
```

Verification Checklist

```
# 1. Backend health
curl -s https://fim.example.com/api/v1/health | jq .

# 2. Login (get token)
TOKEN=$(curl -s -X POST https://fim.example.com/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"FIMAdmin@2024!"}' \
  | jq -r .access_token)
echo "Token: ${TOKEN:0:20}..."

# 3. Check MFA status
curl -s https://fim.example.com/api/v1/mfa/status \
  -H "Authorization: Bearer $TOKEN" | jq .

# 4. Frontend loads
curl -s https://fim.example.com/ | head -20

# 5. Database connectivity
psql -h localhost -U fim_user -d fim_db -c "SELECT COUNT(*) FROM fim.users;"
```

Post-Installation Configuration

Enable MFA for All Users

Recommended in production:

```
psql -U fim_user -d fim_db << 'SQL'
-- Optional: require MFA for admin users
UPDATE fim.users
SET mfa_enabled = true
WHERE role IN ('admin', 'analyst');
SQL
```

Configure Backup Retention

Edit `/usr/local/bin/fim-backup.sh`:

```
KEEP_BACKUPS=2          # Keep only 2 most recent
DB_DUMP_OPTIONS="--compress=9"  # Compress backups
BACKUP_PATH=/opt/fim/backups
```

Setup daily backups:

```
sudo crontab -e
# Add: 0 2 * * * /usr/local/bin/fim-backup.sh
```

Configure Report Scheduling

Reports are generated daily at 2 AM. Customize in `/usr/local/opt/fim/app/services/report_scheduler.py`:

python
```
SCHEDULE_HOUR = 2  # 2 AM UTC
SCHEDULE_MINUTE = 0
```

Troubleshooting

Issue: "Database has no tables — app fails immediately"

Solution: Run migrations

```
source /usr/local/opt/fim/venv/bin/activate
cd /usr/local/opt/fim
python3 << 'PYEOF'
import asyncio
from app.core.database import db_manager
from app.models.models import Base

async def init_db():
    async with db_manager.engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    print("☑ Tables created")

asyncio.run(init_db())
PYEOF
```

Issue: "Can't log in after fresh install"

Solution: Create admin user

```
# See "Create First Admin User" section above
```

Issue: "control.sh crashes on startup"

Solution: Create logs directory

```
mkdir -p /opt/fim/logs
touch /var/log/fim-security.log
sudo chown -R fim:fim /opt/fim/logs /var/log/fim-*.log
```

Issue: "App ignores .env, all config missing"

Solution: Add EnvironmentFile to systemd unit

```
# Edit /etc/systemd/system/fim-backend.service
# Add line: EnvironmentFile=/usr/local/opt/fim/.env
sudo systemctl daemon-reload
sudo systemctl restart fim-backend
```

Issue: "Nginx serves 404 for all UI"

Solution: Build and deploy frontend

```
cd /usr/local/opt/fim/frontend
npm run build
sudo cp -r dist/* /opt/fim/web/
sudo chown -R nginx:nginx /opt/fim/web
```

Issue: "pip install cryptography fails"

Solution: Use Python 3.11+ and install build deps

```
python3.11 -m pip install --upgrade cryptography
# Or on RHEL:
sudo yum install -y python3-devel openssl-devel
pip install cryptography
```

Issue: "RT integration silently fails"

Solution: Verify RT credentials in .env

```
grep -E "^RT_" /usr/local/opt/fim/.env
# Should have: RT_URL, RT_USERNAME, RT_PASSWORD, RT_API_ENDPOINT
```

Test RT connection:

```
python3 << 'PYEOF'
import os
import requests
from requests.auth import HTTPBasicAuth

rt_url = os.getenv('RT_URL')
rt_user = os.getenv('RT_USERNAME')
rt_pass = os.getenv('RT_PASSWORD')

response = requests.get(
  f"{rt_url}/REST/2.0/queues",
  auth=HTTPBasicAuth(rt_user, rt_pass)
)
print(f"RT Status: {response.status_code}")
print(response.text[:200])
PYEOF
```

Known Limitations & Workarounds

Issue	Impact	Workaround
No `alembic upgrade head` step	Manual DB table creation required	Run migrations manually (see above)
No first-admin bootstrap	Can't log in after install	Create admin user manually
`/opt/fim/logs/` not auto-created	control.sh crashes	`mkdir -p /opt/fim/logs`
`EnvironmentFile` missing from systemd	App ignores `.env`	Add to service file and reload
Frontend not built/shipped	Nginx serves 404	Run `npm run build`
Python version not pinned	Build failures on Python 3.8	Explicitly use Python 3.11+
RT credentials not in `.env.example`	Ticket publishing fails silently	Add RT_* variables to `.env`
No disk space monitoring	Disk fills up unexpectedly	Enable `/etc/cron.d/fim-disk-cleanup`

Production Deployment Checklist

- PostgreSQL 15+ installed and running
- `.env` file configured with all required variables
- First admin user created
- SSL certificates installed (Let's Encrypt or CA)
- Nginx configured and tested
- Backend service running (`systemctl status fim-backend`)
- Frontend built and deployed to `/opt/fim/web/`
- Backup script configured and tested
- Daily cron jobs scheduled
- Firewall rules updated (allow 443, 22, restrict 5432)
- Disk space monitoring enabled
- Log rotation configured (`/etc/logrotate.d/fim`)
- MFA enabled for admin users
- Security hardening completed (CSP headers, CSRF, mTLS if needed)

Support & Documentation

- **API Docs:** `https://fim.example.com/docs`
- **OpenAPI Spec:** `https://fim.example.com/openapi.json`
- **Logs:** `/var/log/fim-backend.log`, `/var/log/fim-security.log`
- **Database:** PostgreSQL `fim_db` schema `fim`
- **Backend Code:** `/usr/local/opt/fim/app/`
- **Frontend Code:** `/usr/local/opt/fim/frontend/`

Version Information

- **FIM Version:** Enterprise 2.0+
- **Python:** 3.8+ (tested on 3.11)
- **PostgreSQL:** 15+
- **Node.js:** 18+ (for frontend build)
- **FastAPI:** 0.104+
- **SQLAlchemy:** 2.0+